

25MML0014
ANUSREE V

Video processing

TASK 1

```
25MML0014
ANUSREE V

import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions

# Load pretrained MobileNetV2
model = MobileNetV2(weights='imagenet')

import cv2
import numpy as np

def extract_frames(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Error: Cannot open video")
        return np.array([])

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    # Handle videos where frame count is 0
    if total_frames > 0:
        step = max(1, total_frames // max_frames)
    else:
        step = 1
```

```
    else:
        step = 1

    count = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if count % step == 0:
            frame = cv2.resize(frame, (224, 224))
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) # Important fix
            frames.append(frame)

        count += 1

        if len(frames) >= max_frames:
            break

    cap.release()
    print("Extracted frames:", len(frames))
    return np.array(frames)

def classify_video(video_path):
    frames = extract_frames(video_path)

    if len(frames) == 0:
        print("No frames extracted!")
        return

    frames = preprocess_input(frames)
    predictions = model.predict(frames)
```

```

print("Final Prediction:", top_pred[0][0][1])

# Replace with your video path
video_path = "real_world_demo_video.mp4"
classify_video(video_path)

```

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224.h5
14536120/14536120 ————— 0s 0us/step
Extracted frames: 20
1/1 ————— 2s 2s/step
Downloading data from https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_index.json
35363/35363 ————— 0s 0us/step
digital_clock : 63.84 %
traffic_light : 9.13 %
switch : 1.43 %
computer_keyboard : 1.32 %
matchstick : 0.85 %
Final Prediction: digital_clock

```

Mobile net is detecting digital clock, computer key board, match stick which was not presented in that list, so its giving wrong predictions

WHY

1. **Trained Only on ImageNet Classes** MobileNetV2 is trained on the ImageNet dataset, which contains only 1000 predefined object categories. If an object in the video does not belong to these categories, the model is forced to choose the closest matching class, even if it is incorrect.
2. **No Unknown or Background Class** The softmax output layer of MobileNetV2 always produces a probability distribution across all known classes. Since there is no "unknown" or "none of the above" class, the model must output a label even when the object is unfamiliar.

WHY

1. **Trained Only on ImageNet Classes** MobileNetV2 is trained on the ImageNet dataset, which contains only 1000 predefined object categories. If an object in the video does not belong to these categories, the model is forced to choose the closest matching class, even if it is incorrect.
2. **No Unknown or Background Class** The softmax output layer of MobileNetV2 always produces a probability distribution across all known classes. Since there is no "unknown" or "none of the above" class, the model must output a label even when the object is unfamiliar.
3. **Frame-Based Classification Without Context** The model classifies each video frame independently as a static image. Important contextual information and relationships between frames are ignored, leading to misclassification of partial or unclear objects.
4. **Averaging Predictions Amplifies Errors** Averaging predictions across frames can strengthen incorrect predictions if similar false patterns appear repeatedly due to motion blur, lighting changes, or background textures.
5. **Domain Mismatch Between Training and Real-World Videos** ImageNet images are high-quality, well-centered, and contain clear objects, whereas real-world videos have noise, motion blur, and complex backgrounds. This domain shift causes inaccurate predictions.

print the shape of frames

```

extracted_frames = extract_frames(video_path)
print("Shape of frames:", extracted_frames.shape)
print("First frame (pixels):")
print(extracted_frames[0])

```

```

Shape of frames: (20, 224, 224, 3)
First frame (pixels):
[[[233 206 133]

```

```

Shape of frames: (20, 224, 224, 3)
First frame (pixels):
...
[[233 206 133]
 [233 206 133]
 [233 206 133]
 ...
 [233 206 133]
 [233 206 133]
 [233 206 133]]

[[233 206 133]
 [233 206 133]
 [233 206 133]
 ...
 [233 206 133]
 [233 206 133]
 [233 206 133]]

[[233 206 133]
 [233 206 133]
 [233 206 133]
 ...
 [233 206 133]
 [233 206 133]
 [233 206 133]]

...

[[ 46 48 45]
 [ 46 48 45]
 [ 46 48 45]
 ...
 [ 46 48 45]
 [ 46 48 45]
 [ 46 48 45]]

[[ 46 48 45]
 [ 46 48 45]
 [ 46 48 45]

```

```

▶ extracted_frames = extract_frames(video_path)
  extracted_frames = np.array(extracted_frames)

print("Shape of frames:", extracted_frames.shape)

import matplotlib.pyplot as plt
plt.imshow(extracted_frames[0])
plt.axis("off")
plt.show()

```

... Shape of frames: (25, 224, 224, 3)



Print frames as image

```
[ ] ▶ import numpy as np
import matplotlib.pyplot as plt

# Extract frames
extracted_frames = extract_frames(video_path)

# Ensure numpy array
extracted_frames = np.array(extracted_frames)

print("Shape of frames:", extracted_frames.shape)

# Number of frames to show
num_frames_to_show = min(12, len(extracted_frames))

# Grid size
cols = 4
rows = int(np.ceil(num_frames_to_show / cols))

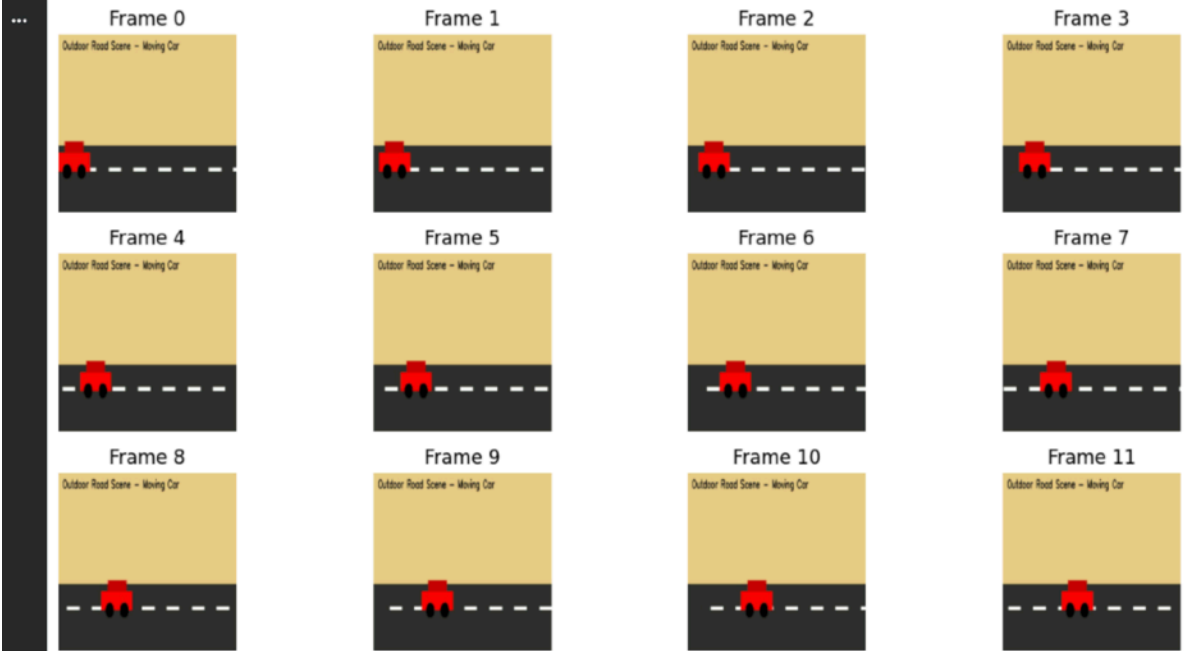
plt.figure(figsize=(12, 6))

for i in range(num_frames_to_show):
    plt.subplot(rows, cols, i + 1)
    plt.imshow(extracted_frames[i])
    plt.title(f"Frame {i}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

... Shape of frames: (25, 224, 224, 3)

Shape of frames: (25, 224, 224, 3)



✓ A CNN + LSTM hybrid model is used for video classification.

```
[4] ▶ import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import TimeDistributed, LSTM, Dense, Dropout

# -----
# PARAMETERS
# -----
IMG_SIZE = 224
MAX_FRAMES = 20

NUM_CLASSES = 2
CLASS_NAMES = ["no_car", "car"]

# -----
# FRAME EXTRACTION
# -----
def extract_frames(video_path, max_frames=MAX_FRAMES):
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Error: Cannot open video")
        return None

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    step = max(1, total_frames // max_frames)

    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    step = max(1, total_frames // max_frames)

    count = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if count % step == 0:
            frame = cv2.resize(frame, (IMG_SIZE, IMG_SIZE))
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame)

        count += 1
        if len(frames) >= max_frames:
            break

    cap.release()

    if len(frames) == 0:
        return None

    # Padding if frames < max_frames
    while len(frames) < max_frames:
        frames.append(frames[-1])

    return np.array(frames)

# -----
# MODEL BUILDING
# -----
cnn = MobileNetV2(
    weights="imagenet",
    include_top=False,
    pooling="avg",
    input_shape=(IMG_SIZE, IMG_SIZE, 3))
```

```
▶ total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
step = max(1, total_frames // max_frames)

count = 0
while True:
    ret, frame = cap.read()
    if not ret:
        break

    if count % step == 0:
        frame = cv2.resize(frame, (IMG_SIZE, IMG_SIZE))
        frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        frames.append(frame)

    count += 1
    if len(frames) >= max_frames:
        break

cap.release()

if len(frames) == 0:
    return None

# Padding if frames < max_frames
while len(frames) < max_frames:
    frames.append(frames[-1])

return np.array(frames)

# -----
# MODEL BUILDING
# -----
cnn = MobileNetV2(
    weights="imagenet",
    include_top=False,
    pooling="avg",
    input_shape=(IMG_SIZE, IMG_SIZE, 3))
```

```

    weights="imagenet",
    include_top=False,
    pooling="avg",
    input_shape=(IMG_SIZE, IMG_SIZE, 3)
)

cnn.trainable = False

model = Sequential([
    TimeDistributed(cnn, input_shape=(MAX_FRAMES, IMG_SIZE, IMG_SIZE, 3)),
    LSTM(128),
    Dropout(0.5),
    Dense(64, activation="relu"),
    Dense(NUM_CLASSES, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# -----
# VIDEO PREPARATION
# -----
def prepare_video(video_path):
    frames = extract_frames(video_path)
    if frames is None:
        return None

    frames = preprocess_input(frames)
    return np.expand_dims(frames, axis=0)

# -----

```

```

# PREDICTION
# -----
video_path = "real_world_demo_video.mp4" # your video

X = prepare_video(video_path)

if X is not None:
    preds = model.predict(X)
    class_id = np.argmax(preds)
    confidence = np.max(preds) * 100

    print("Predicted Class:", CLASS_NAMES[class_id])
    print("Confidence:", round(confidence, 2), "%")
else:
    print("Video processing failed.")

```

... Model: "sequential_2"

Layer (type)	Output Shape	Param #
time_distributed_2 (TimeDistributed)	(None, 20, 1280)	2,257,984
lstm_2 (LSTM)	(None, 128)	721,408
dropout_2 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 2)	130

Total params: 2,987,778 (11.40 MB)
 Trainable params: 729,794 (2.78 MB)
 Non-trainable params: 2,257,984 (8.61 MB)
 1/1 — 26s 26s/step
 Predicted Class: no_car
 Confidence: 51.16 %

Confidence: 51.16 %

MobileNetV2 (CNN) is used as a feature extractor for individual video frames.

A TimeDistributed layer applies the same CNN to every frame in the video.

LSTM (Long Short-Term Memory) is used to learn temporal relationships between frames.

The final Dense layer classifies the video into car or no_car.

✓ Why This Model Works

MobileNetV2 captures important spatial features such as car shape, edges, and structure.

Transfer learning allows the model to reuse pretrained knowledge, reducing training time.

LSTM learns motion and continuity, which helps identify a car appearing across frames.

Temporal learning reduces false predictions caused by blur or background noise.

The model predicts only task-specific classes (car / no_car) instead of forcing ImageNet labels.

Task 2

25MML0014

ANUSREE V

[1]

✓ 0s

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from glob import glob
import IPython.display as ipd
from tqdm.notebook import tqdm
```

[3]

✓ 0s

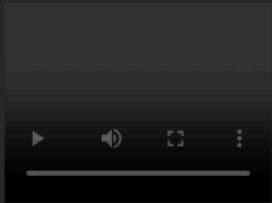
```
#import subprocess
```

[4]

✓ 0s

▶ ipd.Video("026c7465-309f6d33.mp4",width = 200)

...



Open the Video and Read Metadata

[5]

✓ 5s

```
!pip install opencv-python
```

Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python)

[6]

✓ 0s

```
# Load in video capture
import cv2
cap = cv2.VideoCapture('026c7465-309f6d33.mp4')
```

Data Structure of Videos (List Representation)
A video is read frame-by-frame using cv2.VideoCapture().
Each frame is stored as a NumPy array.
All frames are appended into a Python list.
Finally, the list can be converted into a NumPy array for ML processing.

[7]

✓ 0s

```
# Total number of frames in video
cap.get(cv2.CAP_PROP_FRAME_COUNT)
```

2398.0

[8]

✓ 0s

```
# Video height and width
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
print(f'Height {height}, Width {width}')
```

```
7] # Total number of frames in video
   cap.get(cv2.CAP_PROP_FRAME_COUNT)

2398.0

8] # Video height and width
   height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
   width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
   print(f'Height {height}, Width {width}')

Height 720.0, Width 1280.0

9] # Get frames per second
   fps = cap.get(cv2.CAP_PROP_FPS)
   print(f'FPS : {fps:0.2f}')

FPS : 59.94

10] cap.release()

Pull the image from the first frame

11] cap = cv2.VideoCapture('026c7465-309f6d33.mp4')
   ret, img = cap.read() #will return TRUE everytime we call this function till
   print(f'Returned {ret} and img of shape {img.shape}')

Returned True and img of shape (720, 1280, 3)
```

OpenCV Uses BGR Color Format

OpenCV stores images and video frames in BGR (Blue, Green, Red) channel order instead of RGB.

This design choice comes from OpenCV's origins in C/C++, where BGR was commonly used in low-level image processing libraries.

Each frame read using OpenCV is a NumPy array with shape (height, width, 3).

The three channels represent Blue, Green, and Red intensity values in that order.

Many deep learning models and visualization libraries expect RGB format, so direct usage of BGR images may cause incorrect color interpretation.

To ensure correct visualization or model input, OpenCV images are usually converted from BGR to RGB using `cv2.cvtColor()`.

Matplotlib Uses RGB Format

+ Code

+ Text

Matplotlib uses the RGB color format for displaying images, where pixel values are arranged in the order of Red, Green, and Blue channels. This format is widely accepted in visualization libraries and deep learning frameworks. When an image is displayed using `plt.imshow()`, Matplotlib assumes the input image follows RGB channel ordering. However, since OpenCV reads images in BGR format, displaying them directly in Matplotlib results in incorrect color representation. Therefore, it is necessary to convert images from BGR to RGB before visualization to ensure accurate color rendering.

Loading the First Image

```
[12] plt.imshow(img) #opencv loads color in BGR format, matplotlib lib expects RGB format
```

```
<matplotlib.image.AxesImage at 0x7f3ae6686600>
```



```
## Helper function for plotting opencv images in notebook
def display_cv2_img(img, figsize=(10, 10)):
    img_ = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    fig, ax = plt.subplots(figsize=figsize)
    ax.imshow(img_)
    ax.axis("off")
```

```
display_cv2_img(img)
```



[15]

✓ 0s

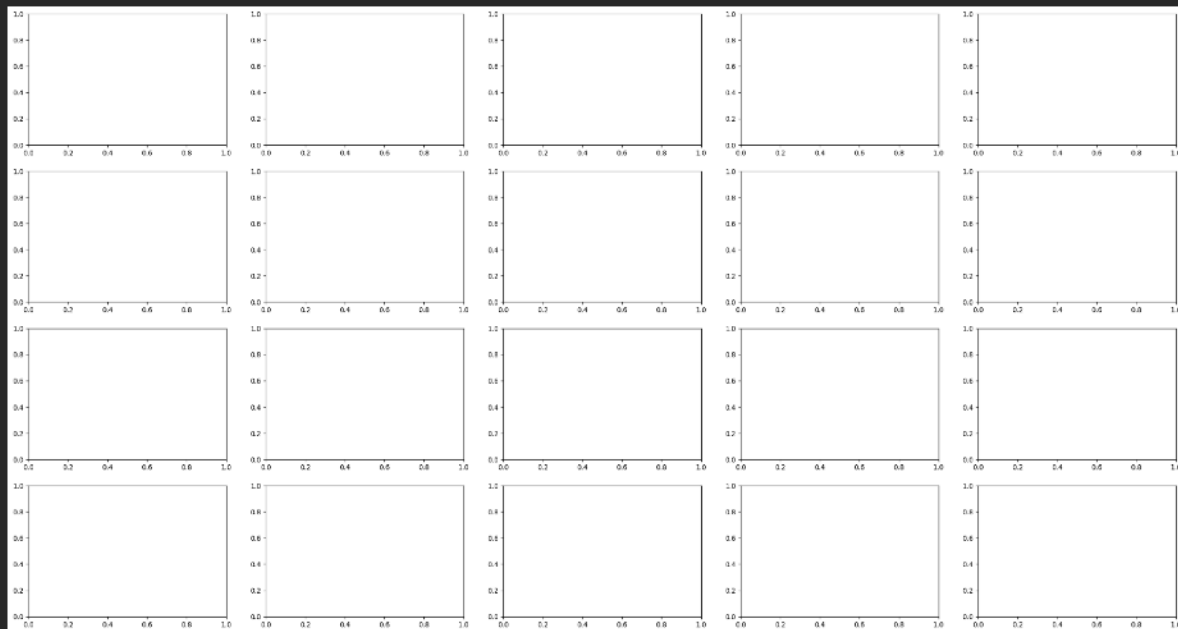
```
cap.release()
```

Display Multiple Frames from the Video

[16]

✓ 2s

```
fig, axes = plt.subplots(5, 5, figsize=(30, 20))  
plt.show()
```

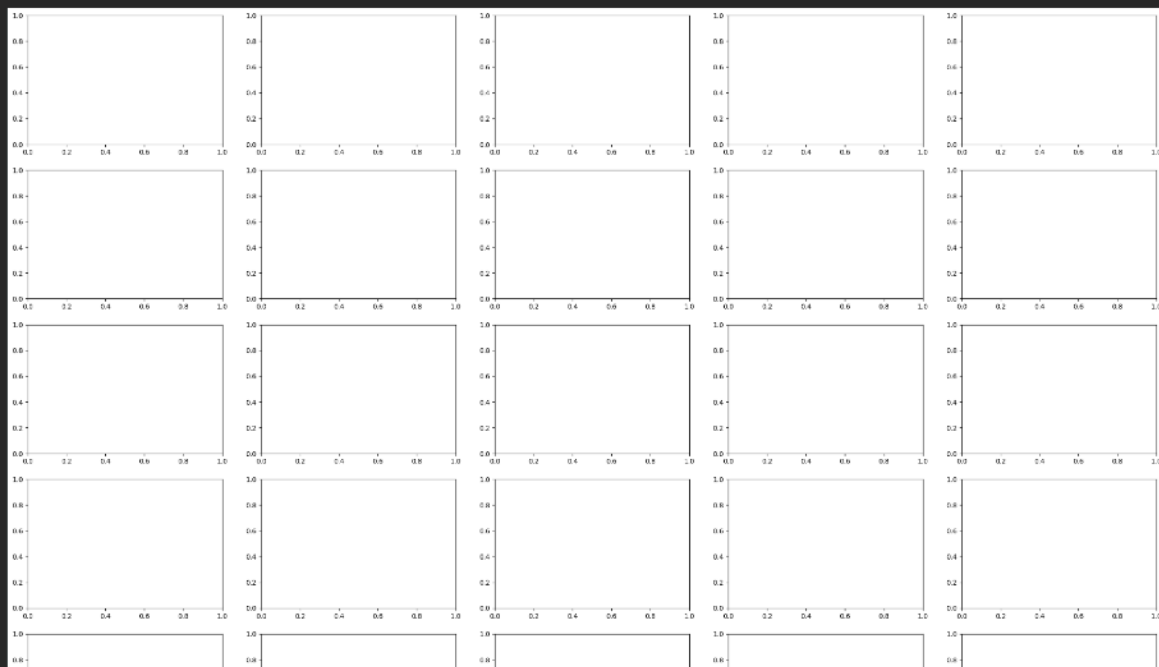


[17]

✓ 2s

```
fig, axes = plt.subplots(5, 5, figsize=(30, 20))  
axes = axes.flatten()  
plt.show()
```

```
# Without flatten → 2D grid (row, column)  
# With flatten → 1D list (single index)
```



```

18]
24s
import cv2
fig, axes = plt.subplots(5, 5, figsize=(30, 20))
axes = axes.flatten()

cap = cv2.VideoCapture("026c7465-309f6d33.mp4")
n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

img_idx = 0
for frame in range(n_frames):
    ret, img = cap.read()
    if ret == False:
        break
    if frame % 100 == 0:
        axes[img_idx].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        axes[img_idx].set_title(f'Frame: {frame}')
        axes[img_idx].axis('off')
        img_idx += 1

plt.tight_layout()
plt.show()
cap.release()

```

#frame is divisible by 100 - Note the modulo operator

#automatically adjusts the spacing between the plots



✓ Video Quality Dimensions – Resolution Levels

1. 480p (SD – Standard Definition)

- Resolution: **854 × 480 pixels**
- Low image clarity and detail
- Requires less storage and bandwidth
- Suitable for basic monitoring where fine details are not required
- Limited accuracy for object detection in surveillance

2. 720p (HD – High Definition)

- Resolution: **1280 × 720 pixels**
- Moderate image quality with clearer object shapes
- Balanced choice between quality and storage
- Commonly used in entry-level surveillance cameras
- Suitable for basic vehicle and person detection

3. 1080p (Full HD)

- Resolution: **1920 × 1080 pixels**
- High clarity and sharp details
- Enables accurate object detection and recognition
- Widely used in modern CCTV and traffic surveillance
- Requires higher storage and processing power

4. 1440p (2K / Quad HD)

- Resolution: **2560 × 1440 pixels**

4. 1440p (2K / Quad HD)

- Resolution: **2560 × 1440 pixels**
- Very high image detail and sharpness
- Suitable for large-area surveillance and zoom-based analysis
- Improves detection accuracy for distant objects
- Requires high bandwidth, storage, and GPU resources

What are Annotations?

Annotations are labels or markings added to images or videos to describe the content.

They provide ground truth information used for training and evaluating machine learning models.

In computer vision, annotations identify what object is present, where it is located, and what action is happening.

Common types of annotations include class labels, bounding boxes, segmentation masks, and keypoints.

In video data, annotations may also include temporal information, such as start and end frames of an activity.

Annotations are essential for supervised learning, especially in object detection, tracking, and activity recognition tasks.

How Annotations Are Used in Surveillance

Annotations are used to label objects such as vehicles, people, or suspicious items in surveillance videos.

They provide ground truth data for training object detection and tracking models.

Bounding box annotations help systems locate and track moving objects across frames.

Temporal annotations define when an event starts and ends, such as intrusion or loitering.

Annotated data improves the accuracy of behavior and activity recognition systems.

Surveillance systems use annotations to generate alerts when predefined rules are violated.

Annotations support video analytics, such as counting vehicles or monitoring restricted areas.

How Annotations Improve Surveillance Systems

Annotations provide accurate ground truth data, which helps surveillance models learn correct object and event patterns.

They improve the accuracy of object detection, such as identifying cars, people, or intrusions.

Annotated videos enable reliable object tracking across multiple frames and cameras.

Temporal annotations help systems understand event duration, such as loitering or unauthorized entry.

High-quality annotations reduce false alarms by teaching the system what is normal and abnormal behavior.

Annotations support activity recognition, allowing systems to detect suspicious or illegal actions.

They help in building automated alert systems for real-time surveillance monitoring.

Check Resolution

```
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
```